

STCF: Shortest Time to Completion First

Implemented rules and examples for Scheduling Studio. These describe the current visualizer, including its deterministic tie and boundary ordering.

Model and terminology

The visualizer uses one CPU, discrete ticks, and CPU-only processes. A process has an arrival time and a service time. Service is the total CPU work required to finish; remaining service is the work still needed. Waiting does not consume service. There is no I/O, blocking, or context-switch cost in the normal UI.

A snapshot at time t shows the state after the scheduling decisions at t and before execution during $[t, t + 1)$. If a process runs during that interval, its remaining service decreases by one. Completion is recorded at the following boundary. Finished processes remain in the metrics table but leave the CPU and ready queue.

1. Compare remaining CPU service

Whenever selection is needed, choose the ready process with the least remaining CPU work. Original service length is not the comparison after a process has run. This policy is also commonly called shortest remaining time first.

Example: A originally needed eight ticks but has only two remaining. B needs three. A has the shorter remaining service.

2. Admit arrivals before checking preemption

Arrivals join the ready queue at their arrival boundary, in input order for simultaneous arrivals. Consider all ready processes when deciding who should run next.

Example: B needs two ticks and C needs one; both arrive together while A has four left. C is the shortest candidate, so there is one preemption of A followed by selection of C.

3. Preempt only for strictly shorter remaining service

If a ready process has less remaining service than the runner, return the runner to the back of the ready queue and select the best process. Preserve its remaining service and previously recorded response/waiting history. There is no priority demotion or budget limit.

The displayed queue keeps insertion order; the selected process may come from anywhere in it. New arrivals are admitted before the interrupted runner is appended.

Example: A has four service ticks remaining when B arrives needing two. B preempts A; A still needs four ticks when it resumes.

4. Keep the current runner on an equal-service tie

An equal remaining service is not enough to cause preemption. This rule applies even if another equal-length ready process would win the usual dispatch tie-breaker.

Example: A starts at zero needing five ticks. At time 2, it has three left. B arrives at 2 needing three ticks. A keeps the CPU.

5. Use arrival and input order for dispatch ties

When the CPU is free and multiple ready processes share the smallest remainder, choose the earlier arrival; if those arrivals tie, use input order. Do not sort alphabetically by ID or choose solely by the displayed queue position.

Example: C and B are both ready with two ticks left. B arrived first, so B wins this dispatch even if C appears before B in the displayed queue.

6. Complete first, then admit, compare, and dispatch

At each boundary:

1. Remove the runner if it has finished.
2. Append this boundary's arrivals.
3. If a runner remains, preempt it only if the best ready remainder is strictly smaller.
4. If the CPU is free, dispatch the shortest ready remainder using the tie rules.
5. Record the boundary, then execute the next tick unless all work has finished.

A completed process is never preempted or requeued. A same-time arrival competes immediately for the freed CPU.

Example: A finishes at 4, B waits with three ticks remaining, and C arrives at 4 with one. C starts at 4.

7. Use no quantum, boosts, or aging

STCF has no turn-length budget or per-priority allotment. Repeated short arrivals may delay a longer process. Waiting time does not override the remaining-service comparison. Record idle ticks when no process is ready, and finish when all accepted processes have completed.

Example: With one process, it runs continuously from arrival to completion. With a later strictly shorter arrival, it can be interrupted and subsequently resume.

Worked example

The following independent example uses input order A, B, C:

Process	Arrival	Service
A	0	5
B	1	2
C	1	1

Interval	CPU	Reason
0–1	A	Only A is ready.
1–2	C	C arrives with one tick, shorter than B's two and A's four remaining.
2–4	B	B's two ticks are shorter than A's four.
4–8	A	A resumes its remaining four ticks.

Per-tick CPU sequence: A C B B A A A A. At time 8, all processes are finished.

Process	Response	Waiting	Turnaround
A	0	3	8
B	1	1	3
C	0	0	1

Metrics and display

- **Response time:** first CPU start minus arrival time. It is recorded once and remains unchanged afterward.
- **Waiting time:** total ticks spent ready but not running. Time before arrival is excluded. At completion, waiting equals turnaround minus original service.
- **Turnaround time:** completion time minus arrival time. It remains unavailable until the process finishes.
- **Averages:** waiting includes every listed process, including future processes with zero waiting. Response averages include only processes that have started; turnaround averages include only finished processes. An unavailable value is shown as a dash. Non-integer averages are displayed to one decimal place.
- **Cards and timeline:** `N service left` is remaining CPU work. A timeline cell at `t` identifies the process running during `[t, t + 1)`, or idle. Animation and reverse playback visualize the same recorded schedule and do not change its decisions.

Input limits and recovery

Use 1–50 processes. IDs must be nonempty and unique after trimming whitespace and ignoring case. Arrival times must be nonnegative safe whole numbers; service times must be positive safe whole numbers. The complete workload, including idle gaps, must finish by tick 2,000. Finishing exactly at 2,000 is allowed; finishing at 2,001 is rejected.

The numeric editor floors fractional entries and clamps them to the field minimum. JSON imports and direct engine calls must pass integer validation. Invalid edits replace the visualization with a validation message; an invalid JSON import leaves the current scenario unchanged. Editing a valid scenario or changing the policy restarts playback at time zero.